

EJERCICIOS 28/09/26

EJERCICIO 1: Potencia mediante multiplicaciones sucesivas.

```
BasePorExponente1.psc X TrianguloTipoYsubBase.psc* ConversionHexadecimal.psc ContadorDePalabras.psc <sin_titulo>*
1 Algoritmo PrimerEjercicio
2   definir Base, Exponente, Resultado Como Entero //Definimos variables como entero, No decimales.
3   //Solicitamos los valores a potencias
4   ESCRIBIR "Ingresa la Base : "
5   leer Base
6   ESCRIBIR "Ingresa el Exponente: "
7   leer Exponente
8
9   //Operacion
10  Resultado ← Base ↑ Exponente //el simbolo "↑" se pone usando alt + 9 + 4 , en ese orden si es que lo tienes bloqueado.
11
12  ESCRIBIR "El resultado es: ", Resultado //Imprimos Resultado
13 FinAlgoritmo
14
```

PSeInt - Ejecutando proceso PRIMEREJERCICIO

*** Ejecución Iniciada. ***

Ingresa la Base :

> 2

Ingresa el Exponente:

> 5

El resultado es: 32

*** Ejecución Finalizada. ***

☐ No cerrar esta ventana ☐ Siempre visible Reiniciar

Algoritmo

PSeInt - Ejecutando proceso PRIMEREJERCICIO

*** Ejecución Iniciada. ***

Ingresa la Base :

> 3

Ingresa el Exponente:

> 4

El resultado es: 81

*** Ejecución Finalizada. ***

☐ No cerrar esta ventana ☐ Siempre visible Reiniciar

Vencimiento 25 de sept

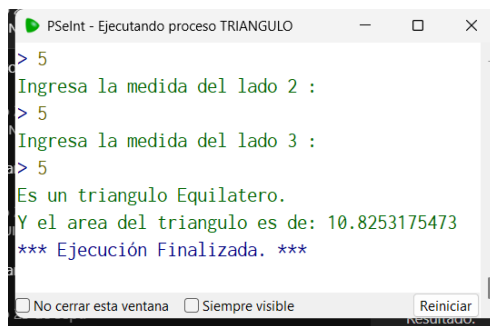
Primeramente definimos nuestras variables que nos piden en este caso el exponente y nuestra base y el resultado

Primero le pedimos la base a el usuario y después le pedimos el exponente, el resultado será la base elevado por el exponente

Y finalmente imprimimos el resultado en la consola.

EJERCICIO 2: Clas Y Area de Un Triangulo

```
1
2 //TIPO: SELECCION
3 Algoritmo Triangulo
4     definir lado1, lado2, lado3, Area, SemiResultado Como Real //Definimos las variables de los 3 lados del triangulo y su area como reales
5     //Solicitamos los datos de los 3 lados //Por que el area del triangulo no puede ser exactamente entero ocupa decimales
6     ESCRIBIR "Ingresa la medida del lado 1 : "
7     leer lado1
8     ESCRIBIR "Ingresa la medida del lado 2 : "
9     leer lado2
10    ESCRIBIR "Ingresa la medida del lado 3 : "
11    leer lado3
12    //Hacemos la operacion
13
14    si (lado1 + lado2 > lado3) y (lado1 + lado3 > lado2) y (lado3 + lado2 > lado1) entonces //AQUI INICIA
15
16        si lado1 = lado2 y lado2 = lado3 Entonces
17            ESCRIBIR "Es un triangulo Equilatero."
18        sino
19            si lado1 = lado2 o lado1 = lado3 entonces
20                ESCRIBIR "Es un Triangulo isoseles."
21            sino
22                si lado1 = lado2 o lado3 = lado3 entonces
23                    ESCRIBIR "Es un triangulo escaleno."
24                FinSi
25            FinSi
26
27        FinSi
28
29        SemiResultado = (lado1 + lado2 + lado3) /2 //PRIMERO SACAMOS EL SemiResultado sumando todos los lados y diviendole entre 2
30
31        //Ahora sacamos la raiz del SemiResultado multiplicando todos los lados menos el SemiResultado esos resultados se multiplican + el SemiResultado
32        //Y DESPUES SE saca su raiz dando como resultado el Area
33        Area = RC(SemiResultado * (SemiResultado - lado1) * (SemiResultado - lado2) * (SemiResultado - lado3))
34
35        //IMPRIMIMOS EL AREA
36        ESCRIBIR "Y el area del triangulo es de: " , Area
37
38    FinSi //TIENE QUE SER AQUI YA QUE SI LO PONEMOS EN OTRO ESTARIAMOS INTERVINIENDO CON OTRO IF
39
40 FinAlgoritmo
41
```



```
N PSeInt - Ejecutando proceso TRIANGULO
C
> 5
Ingresa la medida del lado 2 :
> 5
Ingresa la medida del lado 3 :
> 5
Es un triangulo Equilatero.
Y el area del triangulo es de: 10.8253175473
*** Ejecución Finalizada. ***
[ ] No cerrar esta ventana [ ] Siempre visible [Reiniciar]
```

```
PSeInt - Ejecutando proceso TRIANGULO
> 5
Ingresa la medida del lado 2 :
> 5
Ingresa la medida del lado 3 :
> 6
Es un Triangulo isoseles.
Y el area del triangulo es de: 12
*** Ejecución Finalizada. ***

PSeInt - Ejecutando proceso TRIANGULO
> 4
Ingresa la medida del lado 2 :
> 5
Ingresa la medida del lado 3 :
> 6
Es un triangulo escaleno.
Y el area del triangulo es de: 9.9215674165
*** Ejecución Finalizada. ***
```

Primeramente definimos nuestras variables que nos piden, en este caso los tres lados del triángulo, el área y el resultado de la operación del semiperímetro. Las definimos como reales porque las medidas pueden contener números decimales.

Primero le pedimos al usuario las medidas de los tres lados del triángulo y guardamos cada valor en su respectiva variable.

Después verificamos que las medidas ingresadas realmente puedan formar un triángulo. Para esto comprobamos que la suma de dos lados sea mayor que el tercer lado. Esta condición se realiza con los tres lados para asegurarnos de que sea un triángulo válido.

Una vez comprobado que sí es un triángulo, verificamos qué tipo de triángulo es. Si los tres lados son iguales, es un triángulo equilátero. Si dos lados son iguales, es un triángulo isósceles. Si los lados son diferentes, es un triángulo escaleno.

Después calculamos el SemiResultado, que se obtiene sumando los tres lados y dividiendo el resultado entre 2. Este valor se utiliza para calcular el área del triángulo.

Posteriormente utilizamos la fórmula de Herón para obtener el área. Multiplicamos el SemiResultado por la diferencia entre el SemiResultado y cada uno de los tres lados y finalmente sacamos la raíz cuadrada del resultado.

Finalmente imprimimos en la consola el área del triángulo para que el usuario pueda observar el resultado.

EJERCICIO 3: Conversión de decimal a hexadecimal

```
basevectorExponente.i.psc | triangulo | iporsubase.psc | ConversionHexadecimal.psc | ContadorDePalabras.psc | <sin_titulo>
1  Algoritmo ConversionHexadecimal
2      definir Decimal, Residuo Como Entero //Definimos las variables de el numero a Convertir y el Resultado que seria el residuo
3      definir Hexadecimal, digitoHex Como texto //Los caracteres que convertiran a Hexadecimal
4
5      ESCRIBIR "Ingresa el numero : OJO: NO TIENE QUE SER NEGATIVO:"
6      leer Decimal
7
8      si Decimal < 0 entonces
9          ESCRIBIR "No puede ser numero negativo. Vuelve a intentarlo."
10     sino Hexadecimal ← ""
11         si Decimal = 0 entonces
12             Hexadecimal ← "0"
13
14         sino
15             Mientras decimal > 0 Hacer
16                 Residuo ← decimal Mod 16
17                 segun Residuo hacer
18                     10: digitoHex ← "A"
19                     11: digitoHex ← "B"
20                     12: digitoHex ← "C"
21                     13: digitoHex ← "D"
22                     14: digitoHex ← "E"
23                     15: digitoHex ← "F"
24                 De Otro Modo:
25                     digitoHex ← ConvertirATexto(Residuo)
26             FinSegun
27             Hexadecimal ← digitoHex + Hexadecimal
28             decimal ← Trunc(Decimal / 16)
29         FinMientras
30     FinSi
31
32     ESCRIBIR "El numero Hexadecimal es: ", Hexadecimal
33
34 FinAlgoritmo
```

*** Ejecución Iniciada. ***

Ingresa el numero : OJO: NO TIENE QUE SER NEGATIVO:

> 26

El numero Hexadecimal es: 1A

*** Ejecución Finalizada. ***

PSelnt - Ejecutando proceso CONVERSIONHEXADECIMAL

*** Ejecución Iniciada. ***

Ingresa el numero : OJO: NO TIENE QUE SER NEGATIVO:

> 255

El numero Hexadecimal es: FF

*** Ejecución Finalizada. ***

PSelnt - Ejecutando proceso CONVERSIONHEXADECIMAL

*** Ejecución Iniciada. ***

Ingresa el numero : OJO: NO TIENE QUE SER NEGATIVO:

> 4095

El numero Hexadecimal es: FFF

*** Ejecución Finalizada. ***

Primeramente definimos nuestras variables que nos piden, en este caso Decimal y Residuo como enteros, ya que vamos a trabajar con números enteros. También definimos Hexadecimal y digitohex como texto, porque el resultado hexadecimal puede contener letras de la A a la F.

Primero le pedimos al usuario que ingrese el número decimal que desea convertir y guardamos ese valor en nuestra variable Decimal.

Después verificamos que el número no sea negativo. Si el número es menor que 0, mostramos un mensaje indicando que no puede ser un número negativo y que debe intentarlo nuevamente.

Si el número no es negativo, inicializamos la variable Hexadecimal como un texto vacío. Después comprobamos si el número ingresado es igual a 0. Si es 0, simplemente asignamos "0" como resultado hexadecimal.

Si el número es mayor que 0, utilizamos un ciclo Mientras que se repetirá mientras Decimal sea mayor que 0. Dentro del ciclo obtenemos el residuo de dividir el número entre 16 utilizando Mod 16. Este residuo nos indica qué dígito hexadecimal debemos utilizar.

Después utilizamos una estructura Segun para determinar qué letra corresponde al residuo. Por ejemplo, si el residuo es 10, corresponde a "A"; si es 11 corresponde a "B" y así sucesivamente hasta el 15, que corresponde a "F". Si el residuo es menor que 10, simplemente convertimos ese número a texto.

Después agregamos el dígito hexadecimal al inicio de nuestra variable Hexadecimal. Se coloca al inicio porque los residuos se obtienen desde el último dígito del número hexadecimal hacia el primero.

Finalmente dividimos el número decimal entre 16 y utilizamos Trunc para quedarnos solamente con la parte entera. Esto permite que el ciclo continúe con el siguiente dígito hasta que Decimal llegue a 0.

Cuando termina el ciclo, mostramos en la consola el resultado utilizando ESCRIBIR, indicando cuál es el número hexadecimal correspondiente al número decimal que ingresamos.

EJERCICIO 4: Contador De palabras

```

1  Algoritmo ContadorDePalabras
2  definir palabra, parrafo Como Caracter
3  definir i, NumPalabras, NumParecidos Como Entero // "i" es el numero de veces que contara los espacios
4  definir palabraActual Como Caracter
5
6  ESCRIBIR "INGRESE EL PARRAFO"
7  leer parrafo
8  ESCRIBIR "Ingrese la palabra a buscar: "
9  leer palabra
10
11  NumPalabras ← 1
12
13
14  //=====
15  Para i ← 1 Hasta Longitud(parrafo) Hacer
16
17      Si Subcadena(parrafo, i, i) = " " Entonces
18          NumPalabras ← NumPalabras + 1
19      FinSi
20
21  FinPara
22  //=====
23
24
25  //=====
26  Si NumPalabras > 10 Entonces //Modificar el valor por 100 cuando sea necesario
27
28      NumParecidos ← 0
29      palabraActual ← ""
30      Para i ← 1 Hasta Longitud(parrafo) Hacer
31
32          Si Subcadena(parrafo, i, i) = " " Entonces
33              Si palabraActual = palabra Entonces
34                  NumParecidos ← NumParecidos + 1
35              FinSi
36              palabraActual ← ""
37          SiNo
38              palabraActual ← palabraActual + Subcadena(parrafo, i, i)
39          FinSi
40
41      FinPara
42      Si palabraActual = palabra Entonces
43          NumParecidos ← NumParecidos + 1
44      FinSi
45
46      ESCRIBIR "El parrafo tiene: ", NumPalabras, " palabras"
47      ESCRIBIR "La palabra tiene ", NumParecidos, " coincidencias."
48
49  SiNo
50
51      ESCRIBIR "No tiene mas de 10 palabras."
52
53  FinSi
54  //=====
55
56  FinAlgoritmo

```

*** Ejecución Iniciada. ***

INGRESE EL PARRAFO

> Hola a todos hoy veremos un poco de la programacion orientada a objetos tambien conocida como POO veremos como crear un punto de venta en una app de escritorio y la vol
remos multiplataforma utilizando un poco de flutter con dart y estructuraremos su base de datos relacional para los usuarios y talvez un poco de mongoDB para productos o
sas por el estilo no lo se ademas espero que nos enseñen un poco de webDesign con react ojala sea asi buenas tardes en este ademas de que tendremos que diseñar un pcoo de
ervidores utilizando un poco de linux talvez un poco de ciberseguridad ojala evitemos backdoors en nuestros sistemas como un ingeniero etico ojala sea asi buenas tardes

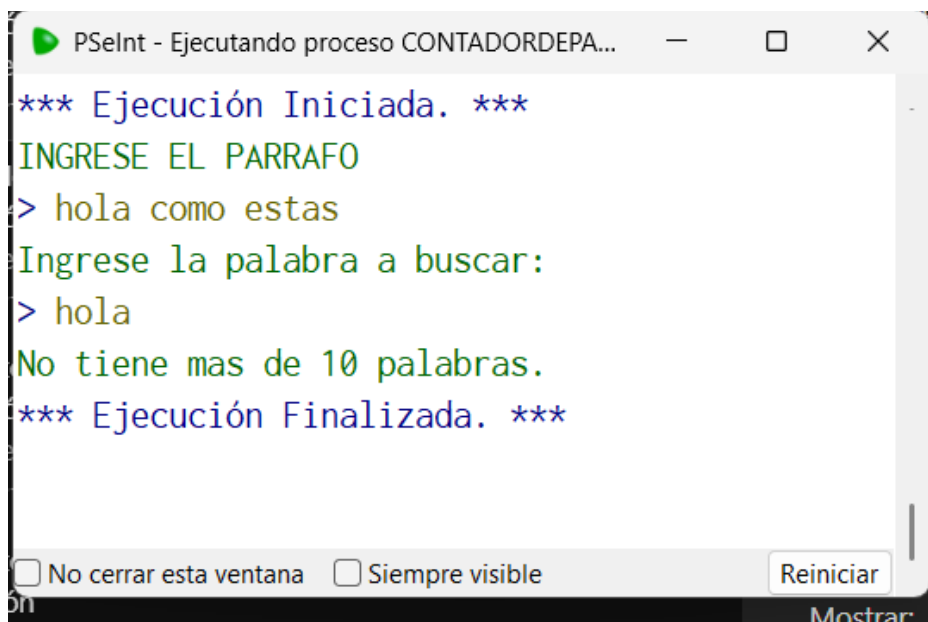
Ingrese la palabra a buscar:

> ojala

El parrafo tiene: 119 palabras

La palabra tiene 3 coincidencias.

*** Ejecución Finalizada. ***



Primeramente definimos nuestras variables que vamos a utilizar. Definimos palabra y parrafo como caracteres porque vamos a trabajar con texto. También definimos i, NumPalabras y NumParecidos como enteros, ya que vamos a utilizarlos para contar y recorrer el párrafo. Finalmente definimos palabraActual como carácter, ya que esta variable nos ayudará a ir formando cada palabra del párrafo.

Primero le pedimos al usuario que ingrese un párrafo y después le pedimos la palabra que desea buscar dentro del párrafo.

Después inicializamos NumPalabras con el valor de 1, ya que el programa comienza contando la primera palabra.

Posteriormente utilizamos un ciclo Para que recorre todo el párrafo utilizando la variable i. La variable i representa la posición de cada carácter dentro del párrafo. En cada posición utilizamos Subcadena para revisar qué carácter se encuentra ahí.

Si el carácter que estamos revisando es un espacio " ", significa que terminó una palabra y comienza otra, por lo que aumentamos NumPalabras en 1. De esta manera podemos saber cuántas palabras contiene el párrafo.

Después verificamos si NumPalabras es mayor que 100. Solamente si el párrafo tiene más de 100 palabras realizamos la búsqueda de la palabra que ingresó el usuario.

Para buscar la palabra, primero ponemos NumParecidos en 0 y palabraActual como texto vacío. Después volvemos a recorrer el párrafo con otro ciclo Para.

Mientras recorremos el párrafo, vamos tomando cada carácter y lo vamos agregando a palabraActual. Cuando encontramos un espacio, significa que terminamos de formar una palabra.

En ese momento comparamos palabraActual con la palabra que el usuario quiere buscar. Si son iguales, aumentamos NumParecidos en 1. Después volvemos a dejar palabraActual vacía para comenzar a formar la siguiente palabra.

Al terminar de recorrer el párrafo, hacemos una última comparación entre palabraActual y palabra, ya que la última palabra del párrafo puede no tener un espacio después de ella.

Finalmente mostramos cuántas palabras tiene el párrafo y cuántas veces apareció la palabra que el usuario buscó.

Si el párrafo no tiene más de 100 palabras, mostramos un mensaje indicando que no cumple con la cantidad necesaria de palabras y no realizamos la búsqueda.

EJERCICIO 5: Binario a decimal

```
BasePorExponente1.psc | TrianguloTipoYsuBase.psc | ConversionHexadecimal.psc | ContadorDePalabras.psc | BinarioADecimal.psc x
1 Algoritmo BinarioADecimal
2 Definir binario Como Caracter
3 Definir i, exponente, decimal Como Entero
4 Definir valido Como Logico
5
6 Escribir "Ingrese un numero binario:"
7 Leer binario
8
9 valido ← Verdadero
10 decimal ← 0
11 exponente ← Longitud(binario) - 1
12
13
14
15 //=====
16 Para i ← 1 Hasta Longitud(binario) Hacer
17     Si Subcadena(binario, i, i) ≠ "0" Y Subcadena(binario, i, i) ≠ "1" Entonces
18         valido ← Falso
19     FinSi
20 FinPara
21
22 //=====
23 Si valido = Verdadero Entonces
24     Para i ← 1 Hasta Longitud(binario) Hacer
25         Si Subcadena(binario, i, i) = "1" Entonces
26             decimal ← decimal + 2 ↑ exponente
27         FinSi
28         exponente ← exponente - 1
29     FinPara
30     Escribir "El numero decimal es: ", decimal
31 SiNo
32     Escribir "El numero ingresado no es binario."
33 FinSi
34 //=====
35 FinAlgoritmo
```

PSInt - Ejecutando proceso BINARIOADECIMAL

*** Ejecución Iniciada. ***

Ingrese un numero binario:

> 1010

El numero decimal es: 10

*** Ejecución Finalizada. ***

☐ No cerrar esta ventana ☐ Siempre visible

PSInt - Ejecutando proceso BINARIOADECIMAL

*** Ejecución Iniciada. ***

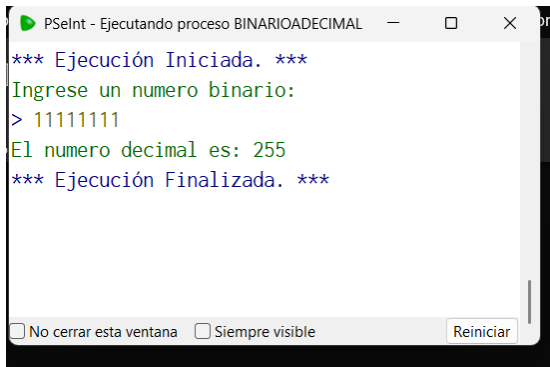
Ingrese un numero binario:

> 11001

El numero decimal es: 25

*** Ejecución Finalizada. ***

☐ No cerrar esta ventana ☐ Siempre visible



```
PSelnt - Ejecutando proceso BINARIOADEDECIMAL
*** Ejecución Iniciada. ***
Ingrese un numero binario:
> 11111111
El numero decimal es: 255
*** Ejecución Finalizada. ***

☐ No cerrar esta ventana ☐ Siempre visible 
```

Primeramente definimos nuestras variables. binario se define como carácter porque trabajaremos con cada dígito del número. i, exponente y decimal se definen como enteros para recorrer el número, manejar las potencias y guardar el resultado. Finalmente, valido se define como lógico para comprobar si el número ingresado es realmente binario.

Primero solicitamos al usuario el número binario y después inicializamos valido como Verdadero, decimal como 0 y exponente con la longitud del número menos 1.

Posteriormente recorremos cada dígito del número utilizando Subcadena y verificamos que solamente contenga 0 o 1. Si aparece otro carácter, cambiamos valido a Falso.

Si el número es válido, recorremos nuevamente sus dígitos. Cuando encontramos un 1, sumamos la potencia de 2 correspondiente al exponente. Después disminuimos el exponente para continuar con la siguiente posición.

Finalmente mostramos el resultado decimal. Si el número no era binario, mostramos un mensaje indicando que el valor ingresado no es válido.